

CSE420

Compiler Design

Topic

Handling Object-Oriented Language Features during Compilation

Prepared by Azmari Sultana

1. From Machine Code to Structural Language Constructs

1.1 What Machine Code Can Actually Do

Every high-level language construct must eventually be expressed using only the capabilities that a real computer's instruction set provides. Modern computer architecture confidently supports exactly four categories of primitive operations, and nothing more:

- Arithmetic and logic operations.
- Single-dimensional array-style load and store operations to and from memory.
- Unconditional jumps to a different instruction in the binary code.
- Conditional jumps to a different instruction in the binary code.

Every structural (non-object-oriented) language feature is translated into a sequence built purely from these four primitives.

1.2 Translating Structural Constructs

Language Construct	Machine-Level Translation
if-else blocks, switch-case	Implemented directly using conditional and unconditional jumps.
while and for loops	Translated exactly like branching code blocks, with one addition: an unconditional jump back to the beginning of the loop after the last instruction of the loop body.
Function calls	Treated as a jump to a different location in the binary where the function's instructions live. The compiler additionally inserts code to push local variables, the caller's return address, and the call parameters onto a stack segment of program memory. When the function returns, a special unconditional jump restores control to the caller, and the pushed variables are restored from the stack.
Multi-dimensional array access	Converted into a single linear (one-dimensional) index access using the array's type information (dimension sizes and element width), and then handled with ordinary load/store instructions.

Figure 1.1 — Branching (if-else and switch-case)

<pre> if (x > 0) \---true---> Block A \---false---> Block B </pre>	<pre> switch (x) --- case 0 ---> Case 0 --- case 1 ---> Case 1 --- case 2 ---> Case 2 --- case 3 ---> Case 3 </pre>
---	---

Figure 1.2 — Loop Translation (while / for)

i < N ?	
True	False
v	v
Block A (Loop Body)	Block B (After Loop)
JUMP A (unconditional, back to condition)	

Figure 1.3 — Function Call via the Runtime Stack

<pre> Caller: sum = add(a, b); </pre>	<pre> Function add(x, y) { ... } </pre>
--	---

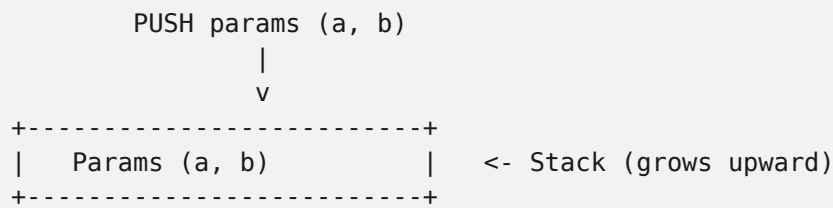


Figure 1.4 — Multi-Dimensional Array Access

2-D Array A[rows][cols]	Row-major linear memory	Register
Select A[1][2] ----->	[0,0][0,1][0,2][0,3][1,0]...	+-----+
	^	
	linear index	+-----+

LOAD : reads data from memory (an address) into a CPU register.
 STORE : writes data from a CPU register into memory (at a given address).

Theory Questions

Q1. List the four primitive capabilities of a machine that every high-level construct must ultimately be translated into.

Ans: (1) Arithmetic and logic operations, (2) single-dimensional array-style load/store to and from memory, (3) unconditional jumps, and (4) conditional jumps to a different instruction in the binary code.

Q2. How does the translation of a while loop differ from the translation of an if-else block?

Ans: Both are built from conditional and unconditional jumps in the same way, but a loop adds one extra unconditional jump at the end of the loop body that jumps back to the beginning of the loop (the condition check), which an if-else block does not need.

Conceptual Questions

Q1. Why does a function call need extra compiler-inserted code beyond a simple jump?

Ans: A plain jump only transfers control; it does not preserve the caller's state. The compiler must also insert code to push local variables, the return address, and the parameters onto the stack before jumping, and to restore them from the stack after the special return jump, so that the caller can resume correctly.

Q2. Why can a 2-D array access ultimately be treated as a 1-D (single-dimensional) load/store?

Ans: Because that is the only kind of array access the machine natively supports. The compiler uses the array's declared type information (its dimension sizes and element width) to compute one linear offset from the multiple indices, after which the access is a normal single-dimensional load or store.

Short Exam-Style Questions

Q1. Name the two kinds of jumps machine code supports.

Ans: Conditional jumps and unconditional jumps.

Q2. What is pushed onto the stack segment during a function call, according to this translation scheme?

Ans: The local variables, the caller's return address, and the function call parameters.

2. Why Object-Oriented Features Need Special Handling

Modern object-oriented programming languages introduce a set of features that have no direct, ready-made equivalent among the four machine-level primitives described in Section 1:

- User-defined types / classes
- Class member functions
- Interfaces and abstract classes
- Field and method inheritance
- Method polymorphism
- Public / private / protected visibility control

A compiler must still reduce every one of these features down to arithmetic/logic operations, single-dimensional load/store, and conditional/unconditional jumps. The remainder of this note works through one standard technique compilers use to achieve this, feature by feature, in the following logical order: (i) how object fields are laid out and accessed, (ii) how visibility modifiers are handled, (iii) how member functions are converted into ordinary functions, and (iv) how inheritance and polymorphism are implemented on top of that conversion.

Theory Questions

Q1. Give two examples of object-oriented language features that have no direct machine-level equivalent.

Ans: Any two of: user-defined classes with member functions, interfaces/abstract classes, field and method inheritance, method polymorphism, and public/private/protected visibility control.

Conceptual Questions

Q1. Why is it accurate to say the compiler 'resolves the mystery of OOP' rather than the machine supporting OOP directly?

Ans: Because the underlying hardware only ever executes arithmetic/logic operations, single-dimensional load/store, and jumps; every OOP construct is purely a source-level abstraction that the compiler must systematically translate down into these same four primitives — the machine itself has no notion of a class, an object, or inheritance.

Short Exam-Style Questions

Q1. State one visibility modifier commonly found in OOP languages.

Ans: Public, private, package, or protected (any one).

3. Fields of User-Defined Types

User-defined types were already available in purely structural languages such as COBOL and C, under names such as structure or record. Without type-specific methods (which OOP classes add), a custom type with fields is nothing more than a variable-grouping feature, and it compiles down to ordinary single-dimensional array accesses in the generated binary code.

3.1 Symbol Table for a Structure

When the compiler encounters a structure/class field-definition, it builds a symbol table recording, for every field:

- **Name** — the field's identifier.
- **Type** — the field's declared type.
- **Width** — the number of bytes the field occupies.
- **Offset** — the field's byte distance from the start of the structure.

Fields are laid out contiguously in declaration order, so the offset of a field equals the sum of the widths of every field declared before it (the first field always has offset 0).

Field-Access Rule: If a variable v of a structure type is located at base address $@v$, then accessing field f generates the three-address code $@v[\text{offset}(f)]$. If field f is itself an array, this expression yields the array's base address, and indexing element i of that array further adds $i \times \text{element_width}$ to reach the desired element.

3.2 Worked Example 1 — Simple Structure: Student

```
struct Student {
    int studentId;
    float cgpa;
    int batch;
    char[20] fullName;
}
```

Assume `int` occupies 4 bytes, `float` occupies 8 bytes, and (for this example) each `char` occupies 2 bytes. The resulting symbol table is:

Name	Type	Width	Offset
studentId	int	4	0
cgpa	float	8	4
batch	int	4	12
fullName	array(20, char)	40	16

Total width of a Student object = $4 + 8 + 4 + 40 = 56$ bytes.

Accessing a simple field

Given an object `adam` stored at base address $@\text{adam}$, the statement:

```
adam.cgpa = 3.85;
```

is translated directly using `cgpa`'s offset (4):

```
@adam[4] = 3.85
```

Accessing an indexed array field

Now consider:

```
ch = adam.fullName[10];
```

`fullName` starts at offset 16. Since each `char` occupies 2 bytes in this example, reaching the 11th element (index 10) requires moving $10 \times 2 = 20$ bytes further from the start of the array. The three-address code is generated in two steps — one temporary to fetch the array's base address, and one final step to index into it:

```
t1 = @adam[16]
ch = t1[20]
```

Verification: $\text{offset}(\text{fullName}) = 16$ (correct, since `studentId`+`cgpa`+`batch` = $4+8+4 = 16$ bytes precede it); index $10 \times \text{width } 2 = 20$ (correct). Both TAC lines check out.

3.3 Worked Example 2 — Nested Structures: Teacher inside Course

A field of a structure can itself be another structure type. Consider:

```

struct Teacher {
    int employeeId;
    char[3] initials;
    char[20] name;
    char[15] position;
}

struct Course {
    int code;
    char[20] name;
    Teacher instructor;
}

```

Using the same convention (int = 4 bytes, char = 2 bytes each), the two symbol tables are:

Name	Type	Width	Offset
employeeId	int	4	0
initials	array(3, char)	6	4
name	array(20, char)	40	10
position	array(15, char)	30	50

Symbol table for struct Teacher (total width = 4 + 6 + 40 + 30 = 80 bytes)

Name	Type	Width	Offset
code	int	4	0
name	array(20, char)	40	4
instructor	Teacher	80	44

Symbol table for struct Course (total width = 4 + 40 + 80 = 124 bytes)

Accessing a doubly-nested, indexed field

Suppose a Course object cse420 exists at base address @cse420, and the program executes:

```
initials = cse420.instructor.initials[2];
```

The chain instructor → initials → index 2 is resolved one level at a time, generating one temporary per level of nesting:

```

t1 = @cse420[44]    // step into 'instructor' (Teacher), offset 44 in Course
t2 = t1[4]          // step into 'initials' (base address), offset 4 in Teacher
initials = t2[4]     // index 2, char width 2 bytes -> 2 x 2 = 4

```

Verification: offset(instructor) = 4 + 40 = 44 ✓; offset(initials) = 4 ✓ (employeeId's width); index 2 × char-width 2 = 4 ✓. All three generated instructions are correct.

3.4 General Rule for Nested Field Access

The pattern generalizes to a chain of any length, a.b.c.d ... :

1. Start from the base address of a, i.e. @a.
2. For each field in the chain, look up its offset in the symbol table of its immediately enclosing type, and generate one instruction $t_k = t_{(k-1)}[\text{offset}]$ (the very first step instead reads $t_1 = @a[\text{offset}]$).
3. Each level of struct nesting therefore contributes exactly one three-address-code instruction.

4. If the final field being accessed is an array and is indexed, generate one last instruction that adds $\text{index} \times \text{element_width}$ to the last obtained base address.

Note: the byte-width assumed for a char (1 byte vs. 2 bytes) varies between examples and exam questions. Always use the width explicitly stated in the problem; the technique for computing offsets and generating TAC is identical regardless of the chosen width.

Theory Questions

Q1. What four pieces of information does a compiler's symbol table store for each field of a structure?

Ans: The field's Name, Type, Width (in bytes), and Offset (byte distance from the start of the structure).

Q2. How is the offset of a field computed?

Ans: It equals the sum of the widths of all fields declared before it in the structure; the first declared field always has offset 0.

Conceptual Questions

Q1. Why does accessing a field inside a nested structure require multiple three-address-code instructions instead of one?

Ans: Because each level of nesting is itself a structure whose fields are only known relative to that structure's own base address. The compiler must first compute the address of the inner structure (using the outer structure's symbol table), then use the inner structure's own symbol table to locate the target field — one indirection/instruction per level of nesting.

Q2. Without type-specific methods, what does a user-defined structure reduce to in the generated binary code?

Ans: A mere variable-grouping feature that is translated into ordinary single-dimensional array accesses, exactly like structures/records in purely structural languages such as C or COBOL.

Short Exam-Style Questions

Q1. In the Student example, what is the total width (in bytes) of a Student object?

Ans: 56 bytes (4 + 8 + 4 + 40).

Q2. In the Course/Teacher example, at what offset does the 'instructor' field sit within a Course object?

Ans: Offset 44 (after code's 4 bytes and name's 40 bytes).

4. Visibility Control: Public / Private / Package / Protected

4.1 Key Principle

There is no such thing as visibility control in the generated binary executable. Visibility control (public, private, package, protected) is strictly a design-time feature of OOP source languages, and the compiler peels it off entirely after semantic analysis is complete.

4.2 How the Compiler Handles Visibility

1. During semantic analysis, for every field or method access appearing anywhere in the program, the compiler checks whether that access is legal according to the public/private/package/protected modifiers declared on the relevant class member.
2. If a violation is found — for example, code in Class A tries to access a private property of Class B — the compiler reports a semantic error at this stage.

3. Once semantic analysis confirms that every access in the program is valid, the compiler's job regarding visibility is finished.
4. During all later code-generation phases, the compiler completely ignores visibility modifiers, generating code exactly as though every member were public — as if the modifiers had never been written.

4.3 Related Compile-Time-Only Checks

The same pattern — validate during semantic analysis, then discard and ignore during code generation — also governs a few related OOP restrictions:

- **final / constant keywords** — enforced (e.g., preventing reassignment) only during semantic analysis; they leave no trace in the generated code.
- **Abstract class instantiation** — the rule that an abstract class cannot be instantiated is validated during semantic analysis and is likewise ignored during later code-generation phases.

Theory Questions

Q1. At which compiler phase are visibility-modifier violations detected?

Ans: During semantic analysis. If an access violates the declared public/private/package/protected modifiers, a semantic error is reported at that stage.

Q2. What happens to visibility modifiers once semantic analysis has validated the program?

Ans: They are completely discarded; the compiler generates code for later phases exactly as if every class member were public.

Conceptual Questions

Q1. Why is it correct to say visibility control has zero effect on the executable's runtime behaviour?

Ans: Because visibility is purely a compile-time legality check on the source program. Once the compiler has confirmed every access is valid, it removes all visibility information before code generation, so the produced binary contains no visibility checks or restrictions at all — accessing any field 'illegally' at the machine level would work exactly the same as any other access.

Short Exam-Style Questions

Q1. Name one other OOP restriction (besides visibility) that is validated only during semantic analysis and then ignored.

Ans: Either the final/constant keyword restriction, or the rule that an abstract class cannot be instantiated.

5. Class Member Functions: Converting to Global Functions

5.1 The Problem

At the binary code level, the only kind of function available is a regular function: code simply jumps to it and eventually returns. There is no native notion of a function 'belonging' to an object. Yet OOP languages let classes define member functions (methods) that operate on a particular object's data. The compiler must bridge this gap.

5.2 The “this” Parameter Solution

1. During semantic analysis of a method, the compiler builds a symbol table for the method's own scope and classifies every variable it uses into exactly one of three scopes: (a) function parameter scope —

the method's declared parameters; (b) local scope — variables declared inside the method body; (c) class scope — fields/properties of the enclosing class.

2. The compiler rewrites the method as an ordinary global function by inserting one new, implicit first parameter — conventionally named `this`, typed as the enclosing class.
3. Every reference inside the method body to a class-scope field is rewritten to be explicitly qualified as `this.fieldName`, since that field's storage now lives in the object `this` points to, not in the function's own activation record. Parameters and local variables are left unchanged, since they already belonged to the function's own activation record.
4. Every call site that invokes the method on an object is rewritten to match: `obj.method(args)` becomes `method(obj, args)` — the receiver object is passed as an extra first argument.

Rule: invoking a member function of a class object is, at the end, actually invoking a global function with the object passed as the first parameter.

5.3 Worked Example — `Student.isInProbation`

```
public class Student {
    private int id;
    private String name;
    private List semesterGPAs;

    public Boolean isInProbation(float thresholdCGPA) {
        float gpaSum = 0.0;
        foreach (semGPA in semesterGPAs) {
            gpaSum += semGPA;
        }
        float cgpa = gpaSum / semesterGPAs.length();
        if (cgpa > thresholdCGPA) { return false; }
        return true;
    }
}
```

Scope classification determined by the compiler for `isInProbation`:

Variable	Scope
<code>thresholdCGPA</code>	Function parameter scope
<code>gpaSum, semGPA, cgpa</code>	Local (function-body) scope
<code>semesterGPAs</code>	Class scope

Applying the `this`-parameter rule converts the method into the following global function (class-scope references rewritten to `this.____`):

```
public Boolean isInProbation(Student this, float thresholdCGPA) {
    float gpaSum = 0.0;
    foreach (semGPA in this.semesterGPAs) {
        gpaSum += semGPA;
    }
    float cgpa = gpaSum / semesterGPAs.length();
    if (cgpa > thresholdCGPA) { return false; }
    return true;
}
```

Every call site is rewritten in the same way:

Source form:	<code>Boolean probation = studentA.isInProbation(threshold);</code>
Converted form:	<code>Boolean probation = isInProbation(studentA, threshold);</code>

Theory Questions

Q1. What implicit parameter does the compiler add when converting a member function to a global function, and what is its type?

Ans: It adds this as the new first parameter, typed as the class the method belongs to.

Q2. Into which three scopes does the compiler classify variables used inside a method body?

Ans: Function parameter scope, local (function-body) scope, and class scope.

Conceptual Questions

Q1. Why must class-scope fields be rewritten as `this.fieldName` after conversion, while local variables and parameters are left unchanged?

Ans: Local variables and parameters already live in the function's own activation record and are unaffected by the class-to-function conversion. Class-scope fields, however, are properties of the object the method operates on; once the method becomes a plain global function, those fields are no longer implicitly in scope, so every reference to them must be routed explicitly through the object referenced by this.

Short Exam-Style Questions

Q1. Rewrite the call `obj.doWork(x, y)` as it would appear after member-function conversion.

Ans: `doWork(obj, x, y)`

Practice Problem

Source: Spring 2026, Set A — Question 3

[CO4] [1 + 1 = 2 marks]

Question: “When a compiler converts a class member function to a global function, how does the scope of variables change? Specifically, how are class-level fields accessed differently from local variables inside the method?”

Solution

(a) How the scope changes: the compiler adds one new, implicit first parameter — `this`, typed as the enclosing class — turning the method into an ordinary global function. This does not disturb the scope of the method's own parameters or local variables at all; they continue to live in the function's own parameter scope and local scope exactly as before, since that storage already belonged to the function's own activation record. What changes is that the class's field variables, previously implicitly visible throughout the method body (class scope), are no longer in scope by default inside the converted global function — they must now be reached explicitly through the `this` parameter.

(b) Access difference: local variables and parameters continue to be accessed directly by name, resolved by the compiler to offsets within the function's own activation record, completely unaffected by the conversion. Class-level fields, by contrast, must be rewritten as `this.fieldName` — the compiler resolves the field's offset within the object that `this` points to (using that class's field symbol table, as in Section 3), rather than within the function's own local activation record. For example, the field `semesterGPAs` becomes `this.semesterGPAs` after conversion, while the parameter `thresholdCGPA` and the locals `gpaSum`, `semGPA` and `cgpa` are referenced unchanged.

6. Class Inheritance and Method Polymorphism

Handling inheritance and polymorphism is the most involved part of compiling OOP code, because it must remain correct even when the compiler cannot know, at compile time, the exact runtime type of every object.

6.1 Setup Example: Person and Student

```
class Person {
    String name;
    int NID;
    String address;
    int age;
    String getName() {
        return name;
    }
}

class Student extends Person {
    String department;
    int studentId;
    String getDepartment() {
        return department;
    }
    String getDescription() {
        String desc = name + ", " +
            address;
        return desc;
    }
    int getStudentId() {
        return studentId;
    }
}
```

Student extends Person and overrides getDescription().

6.2 Step 1 — Converting Every Method to a Global Function

Applying the this-parameter rule from Section 5 to both classes produces five global functions in total:

- String getName(Person this)
- String getDescription(Person this)
- String getDepartment(Student this)
- String getDescription(Student this)
- int getStudentId(Student this)

6.3 Step 2 — Method Overriding

Because Student re-declares getDescription() with the same signature, two separate implementations now exist: getDescription(Person this) and getDescription(Student this). The compiler must ensure that calling getDescription() on a Student object always reaches the Student implementation, even if that object is referenced through a Person-typed variable.

6.4 The VTable (Virtual Table)

Definition: a vtable is a per-class table generated by the compiler. Each entry corresponds to one of the class's methods (inherited or its own) and stores the address of the correct implementation to use for objects of that class.

- A method the subclass does not override: its vtable entry simply points to the base class's implementation (inherited unchanged).

- A method the subclass overrides: its vtable entry is replaced to point to the subclass's own implementation.
- A method introduced only in the subclass: a brand-new entry is added, existing only in the subclass's vtable.

Method	Person's VTable Entry Points To
getName	String getName(Person this)
getDescription	String getDescription(Person this)

VTable for class Person

Method	Student's VTable Entry Points To	Status
getName	String getName(Person this)	Inherited, unchanged
getDescription	String getDescription(Student this)	Overridden
getDepartment	String getDepartment(Student this)	New
getStudentId	int getStudentId(Student this)	New

VTable for class Student

6.5 The VPointer and Object Memory Layout

The compiler appends a new, hidden pointer field — conventionally named `vpointer` — to every class. Every object stores in this field a pointer to its own class's vtable: a `Person` object's `vpointer` points to `Person`'s vtable, and a `Student` object's `vpointer` points to `Student`'s vtable.

A subclass object's field memory layout extends its parent's layout: the inherited base-class fields keep their original order and offsets, the subclass's own new fields are appended after them, and the `vpointer` is added as well.

<pre>class Person String name; int NID; String address; int age; int *vpointer --> [Person's VTable]</pre>	<pre>class Student String name; (inherited) int NID; (inherited) String address; (inherited) int age; (inherited) String department; (own) int studentId; (own) int *vpointer --> [Student's VTable]</pre>
---	---

6.6 Dynamic Method Dispatch — Step-by-Step Algorithm

Given a call such as `obj.method(args)`, the compiler does not hard-wire the call to one fixed implementation. Instead it generates code that performs the following steps at runtime:

1. Access the `vpointer` property of the object `obj`.
2. Call a generic global lookup routine, passing in the vtable that `vpointer` references together with the name of the method; this routine returns the memory address of the correct implementation to invoke.
3. Add the object itself as the first ('this') parameter of the call.
4. Add the remaining original parameters exactly as specified in the source program.
5. Call the function at the address obtained in Step 2.

6.7 Why VTables and VPointers Are Necessary

A subclass object can be assigned, at runtime, to a variable declared with the superclass's type. Consequently the compiler cannot know — while generating code — the true runtime type of the object a variable will actually reference; that type is only revealed during program execution. Routing every call through the object's own vpointer defers the decision of which implementation to run until runtime: the vpointer always points to the vtable of the object's actual (not its declared) class, so the correct, type-specific implementation is always the one executed. This indirection is precisely the mechanism that implements dynamic (runtime) polymorphism.

Theory Questions

Q1. What is a vtable, and what does each of its entries store?

Ans: A vtable is a per-class table generated by the compiler; each entry corresponds to one of the class's methods and stores the memory address of the correct implementation of that method to use for objects of that class.

Q2. What is a vpointer, and where is it stored?

Ans: A vpointer is a hidden pointer field the compiler appends to every class; it is stored as part of every object of that class, and it points to that class's own vtable.

Q3. List, in order, the five steps of dynamic method dispatch.

Ans: (1) Access the object's vpointer. (2) Use it, with the method name, to look up the correct implementation's address in the vtable. (3) Add the object as the first (this) parameter. (4) Add the remaining original parameters. (5) Call the function at the address obtained in step 2.

Conceptual Questions

Q1. Why can't the compiler simply decide at compile time which implementation of an overridden method to call?

Ans: Because a subclass object may be assigned to a variable declared with a superclass type, and this assignment happens at runtime. The compiler therefore cannot know the object's actual runtime type while generating code, so it cannot statically bind an overridden call to one fixed implementation; it must defer that decision to runtime via the vpointer/vtable indirection.

Q2. How does a subclass's memory layout relate to its base class's memory layout, and why is this useful?

Ans: A subclass's object layout begins with exactly the same fields, in the same order, as its base class, followed by the subclass's own new fields (and the vpointer). This means a Student object's first fields line up exactly with a Person object's fields, so any code written to operate on a Person object continues to work correctly even when it is actually handed a Student object.

Short Exam-Style Questions

Q1. In the Person/Student example, which method has two separate vtable-pointed implementations, and why?

Ans: getDescription, because Student overrides the version defined in Person, so Person's vtable and Student's vtable point to two different implementations of it.

Q2. Which entries in Student's vtable are new (not present at all in Person's vtable)?

Ans: getDepartment and getStudentId.

Practice Problems

Source: Fall 2025, Set A — Question 1

[CO5] [3 marks]

Question: “Explain, with an example, how a compiler resolves class functions in two classes when they have the same parameters and one class inherits from the other. Furthermore, explain how these functions are correctly called/invoked in the executable code and how function access modifiers impact function calls if they have some impact.”

Solution

Example setup: let Person define getDescription(), and let Student extend Person while also defining its own getDescription() with the same signature (no parameters, returns String) — exactly the case in Section 6.1.

Resolving the same-signature functions: the compiler first converts every method into a global function carrying an added this parameter (Section 5), producing two functions distinguished by the type of this: getDescription(Person this) and getDescription(Student this). Because Student overrides the method, the compiler builds a vtable for each class: Person's vtable entry for getDescription points to getDescription(Person this); Student's vtable entry is overwritten to point to getDescription(Student this) instead. Every non-overridden entry in Student's vtable (such as getName) simply points to the same implementation as Person's vtable.

Correct invocation in the executable: for a call obj.getDescription(), the compiler does not hard-wire the call to one implementation. It generates code that (1) reads obj's vpointer, (2) uses it to reach the vtable of obj's actual runtime class, (3) looks up the getDescription entry to obtain the correct implementation's address, and (4) calls that address, passing obj itself as the first (this) argument. Because the vpointer always reflects the object's true runtime type — even if obj is referenced through a Person-typed variable while actually holding a Student object — this indirection guarantees that the Student implementation runs whenever the object really is a Student. This is dynamic dispatch.

Effect of access modifiers: public/private/protected modifiers have no effect whatsoever on the generated executable code or on how the call above is performed. They are used only during semantic analysis, to verify that every access in the source program is legal; any violation is reported as a compile-time semantic error. Once validated, the compiler discards all modifier information and generates code exactly as if every member were public — modifiers impose no restriction or cost on function calls in the compiled executable.

Source: Spring 2026, Set A — Question 4

[CO5] [1.5 + 1.5 + 2 = 5 marks]

Question: given the nested structures below, and S studentRecord; (int occupies 4 bytes, char occupies 1 byte, @studentRecord is the base address of studentRecord):

```
struct P {
    char grade;
    int marks;
    char section[6];
};

struct Q {
    int courseId;
    P result;
    char room[5];
};

struct R {
    char title[8];
    Q course;
    int semester;
};

struct S {
    int studentId;
    R info;
    char university[12];
};
```

- a. Determine the total offset of the field accessed in `y = studentRecord.info.course.result.section[4];` relative to `@studentRecord`.

- b. Write the equivalent three-address code for the statement in part (a).
- c. Describe how subclasses in inheritance are represented in memory layout during compilation, and explain with an example why a stack might be necessary to implement this.

Solution

Building the symbol tables (Section 3 technique, char = 1 byte, int = 4 bytes):

Struct	Field	Width	Offset
P	grade (char)	1	0
P	marks (int)	4	1
P	section (char[6])	6	5
Q	courseld (int)	4	0
Q	result (P)	11	4
Q	room (char[5])	5	15
R	title (char[8])	8	0
R	course (Q)	20	8
R	semester (int)	4	28
S	studentId (int)	4	0
S	info (R)	32	4
S	university (char[12])	12	36

Width(P) = 1+4+6 = 11 bytes. Width(Q) = 4+11+5 = 20 bytes. Width(R) = 8+20+4 = 32 bytes. Width(S) = 4+32+12 = 48 bytes.

(a) Total offset

Resolving the chain studentRecord → info → course → result → section[4] one level at a time:

- offset(info) in S = 4
- offset(course) in R = 8
- offset(result) in Q = 4
- offset(section) in P = 5
- index 4, char width 1 byte → $4 \times 1 = 4$

Total offset = 4 + 8 + 4 + 5 + 4 = 25 bytes relative to @studentRecord.

(b) Three-address code

Following the nested-field-access rule from Section 3.4, one temporary per level of nesting:

```

t1 = @studentRecord[4]    // step into 'info' (R)
t2 = t1[8]                // step into 'course' (Q)
t3 = t2[4]                // step into 'result' (P)
t4 = t3[5]                // base address of 'section' array
y  = t4[4]                // section[4], 1 byte per char -> 4 x 1 = 4

```

(c) Subclass memory layout and the need for a stack

Memory layout: as described in Section 6.5, a subclass object's field layout extends its base class's layout — the inherited fields are kept, in their original order and offsets, at the start of the subclass's layout, and the subclass's own new fields (plus the vpointer) are appended afterward. This means the base-class portion of a subclass object always sits at the same starting offsets as in a pure base-class object, which is exactly why a subclass object can be safely referenced through a base-class-typed variable.

Why a stack is necessary: computing this layout means the compiler must walk into a nested/inherited type definition while it is still in the middle of computing the offsets of the enclosing type (exactly as in part (a), where the compiler had to descend from S into R, then Q, then P). Before descending into the nested type, the compiler must remember the enclosing type's current symbol table and current running offset so that, once the nested type's own layout has been fully computed, it can resume computing the enclosing type's remaining fields from exactly where it left off. Because record/class definitions can be nested to any depth and are processed by a recursive, top-down traversal, a stack (a last-in-first-out structure) is the natural mechanism for this: the compiler pushes the current table and offset before entering a nested definition, and pops them back to restore the outer context after that definition is finished — correctly handling any depth of nesting or inheritance.